Consistency, Replication, Fault Tolerance & Security



Data-centric & client-centric consistency · Replica placement · CAP theorem · Process resilience · 2PC / 3PC · DFS security

LECTURE 04

01

Consistency Models

Data-centric & client-centric views of consistency

02

Replication

Strategies and placement of replicas

03

CAP Theorem

Consistency, availability, partition tolerance trade-offs

04

Fault Tolerance

Failure types, redundancy, process resilience

05

Failure Detection

Heartbeat and gossip protocols

06

Reliable Communication

Client-server and group communication

07

Commit Protocols

Two-Phase and Three-Phase Commit

08

Security

Access control in distributed file systems

Where We Left Off

- **Distributed system:** independent computers that appear to users as a single coherent system.
- **Middleware:** software layer that hides heterogeneity and provides a uniform interface.
- **Transparency goals:** access, location, migration, replication, concurrency, and failure transparency.
- **Why replicate at all?** improve reliability and performance — but every copy raises a new question: are all copies the same?

Processes → Communication → Naming

Today's question:
Once data is copied across nodes for reliability and speed, how do we keep those copies close enough to "the truth" for applications to trust them?

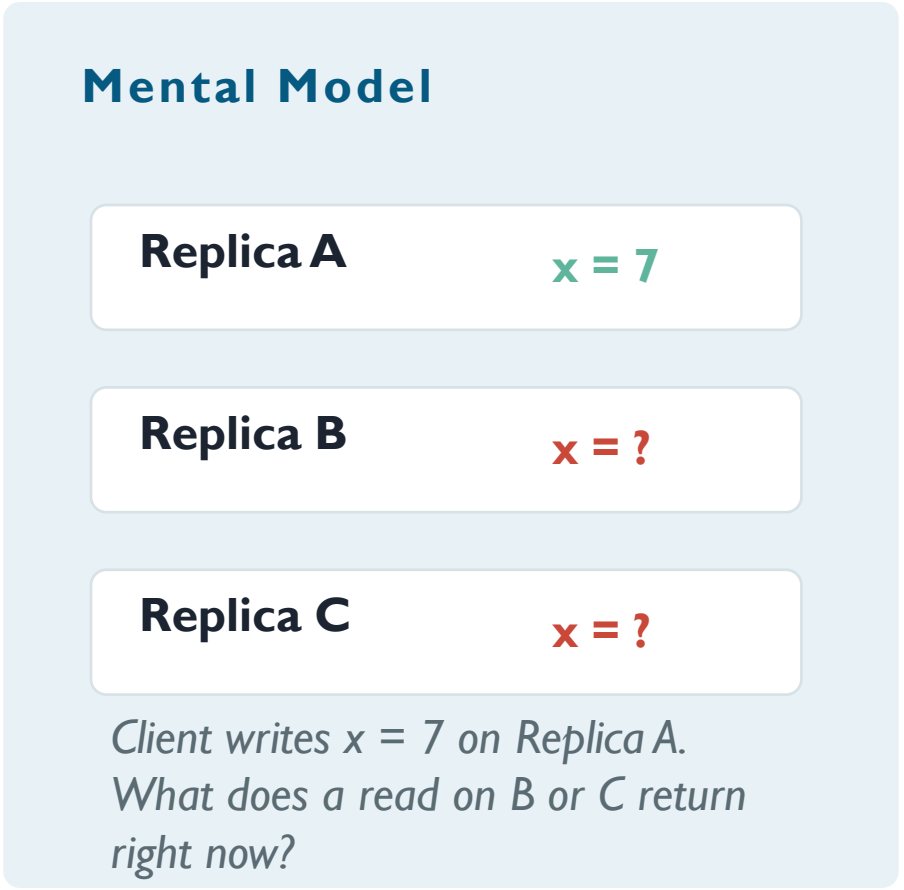
Learning Objectives

- ✓ Distinguish data-centric from client-centric consistency models and give an example of each.
- ✓ Explain replica placement strategies and the trade-offs in choosing where copies live.
- ✓ State the CAP theorem and classify real systems as CP, AP, or CA.
- ✓ Describe failure models and how heartbeat and gossip protocols detect failures.
- ✓ Compare reliable client-server communication with reliable group (multicast) communication.
- ✓ Walk through Two-Phase and Three-Phase Commit and explain why 3PC exists.
- ✓ Identify the core access-control mechanisms used to secure distributed file systems.

Consistency Models

Why Consistency Matters

- **Replication helps performance,** but every replica can be updated independently — copies can drift apart.
- **A consistency model** is a contract between the data store and processes: it specifies what values a read is allowed to return after a sequence of writes.
- **Stronger models** are easier to reason about but costlier to implement (more coordination, more latency).
- **Weaker models** scale better and tolerate partitions, at the cost of applications seeing stale or out-of-order data.



Data-Centric vs. Client-Centric Consistency

Data-Centric Models

- Define consistency from the data store's point of view.
- Concerned with the actual (total or partial) ordering of operations, across ALL processes, on shared data.
- Examples: strict, sequential, causal, eventual consistency.
- Best when many processes concurrently access shared data (e.g., a database, a distributed shared memory).

Client-Centric Models

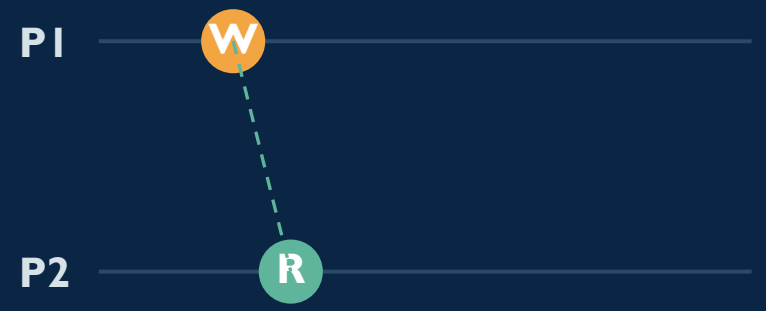
- Define consistency from a single mobile client's point of view.
- Concerned with what ONE process sees across successive accesses, even as it moves between replicas.
- Examples: monotonic reads, monotonic writes, read-your-writes, writes-follow-reads.
- Best for weakly-consistent stores with occasional / lazy replica updates (e.g., a mobile calendar app).

Strict Consistency

- **Rule:** any read returns the value of the most recent write — as if there were a single global clock and a single copy of the data.
- **Requires** absolute global time, which real distributed systems cannot guarantee (network delay, clock skew).
- **Rarely achievable** in practice; it is the theoretical upper bound other models are measured against.

"Any read on a data item x returns a value corresponding to the result of the most recent write on x." — Tanenbaum & Van Steen

Read after write, instantly visible



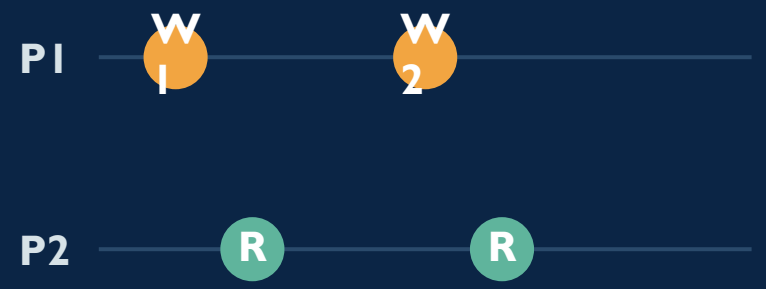
R returns the new value — immediately, no matter how close in time.

Sequential Consistency

- **Rule:** the result of any execution is the same as if operations of all processes were executed in SOME sequential order.
- **Each process's own operations** appear in that order in the sequence they were issued — but ordering across different processes can vary.
- **No global real-time requirement,** so it's implementable — used in some distributed shared-memory systems.

"All processes see the SAME interleaving of operations — but that interleaving does not have to match real (wall-clock) time."

One agreed-upon interleaving



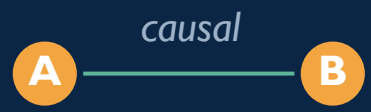
Every process agrees W1 → W2 → reads — same order for all, even if not real-time order.

Causal Consistency

- **Rule:** writes that are causally related must be seen by every process in the same order; concurrent (unrelated) writes may be seen in different orders.
- **"Causally related"** means one write happened-before another — e.g., a reply to a post must appear after the post.
- **Cheaper than sequential consistency** because unrelated operations don't need global agreement.

Example: a forum reply ("B") that references a post ("A") must never be seen by anyone before "A" — but two unrelated posts can appear in any order.

Causally related vs. concurrent writes





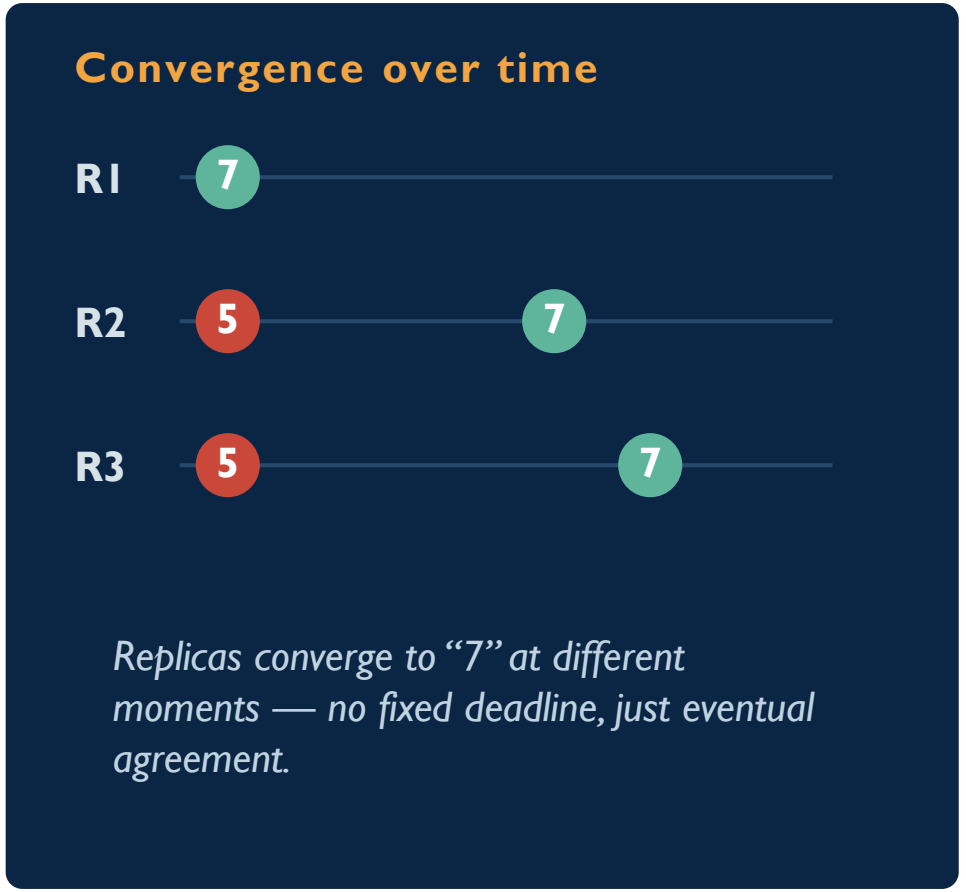
concurrent — order may differ per replica

All processes see A before B. Processes may see C, D in different orders.

Eventual Consistency

- **Rule:** if no new updates are made, all replicas will EVENTUALLY converge to the same value — no guarantee about ordering or timing in between.
- **Optimized for availability & performance,** not immediate agreement — common in DNS, web caches, and many NoSQL stores.
- **Trade-off:** readers can observe stale data for an unbounded (but typically short) period.

"In the absence of new updates, all accesses will eventually return the last updated value." — widely used in large-scale, highly available key-value stores (e.g., DynamoDB-style systems).



PUTTING IT TOGETHER

The Consistency Spectrum

Strict

Real-time order, single copy illusion

Causal

Order only for related ops



STRONGER ← guarantees → WEAKER

Sequential

One global interleaving

Eventual

Convergence, no ordering guarantee

Performance & availability generally increase as guarantees weaken — choosing a model means choosing a point on this line for your workload.

Client-Centric Consistency — Overview

- **Setting:** a mobile client reads/writes through DIFFERENT replicas over time (e.g., moving between Wi-Fi access points).
- **Goal:** guarantee that, from that ONE client's viewpoint, the world still makes sense — even though replicas sync lazily in the background.
- **Four classic guarantees:** monotonic reads, monotonic writes, read-your-writes, writes-follow-reads.
- **These are NOT alternatives to each other** — a system can, and often does, offer several simultaneously.

Scenario

Home Wi-Fi replica
Client connects here at t1

Office Wi-Fi replica
Client connects here at t2

Mobile-data replica
Client connects here at t3

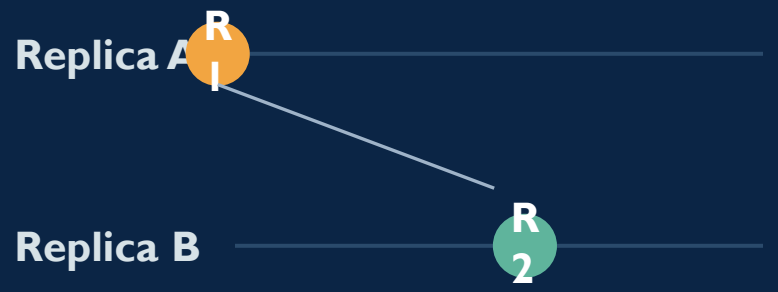
Same user, three different replicas, one continuous session — client-centric models keep it coherent.

Monotonic Reads

- **Guarantee:** if a process reads value v of item x, any later read by that SAME process will never return an older value than v.
- **In other words:** once you've seen an update, you never "go backward in time" on subsequent reads.
- **Violated when** a client reads a stale, un-synced replica after already reading a fresher one.

Example: you see 5 new emails on your laptop; on your phone (a different, lagging replica) you must not see FEWER than those 5 — monotonic reads guarantees you won't.

Illustration

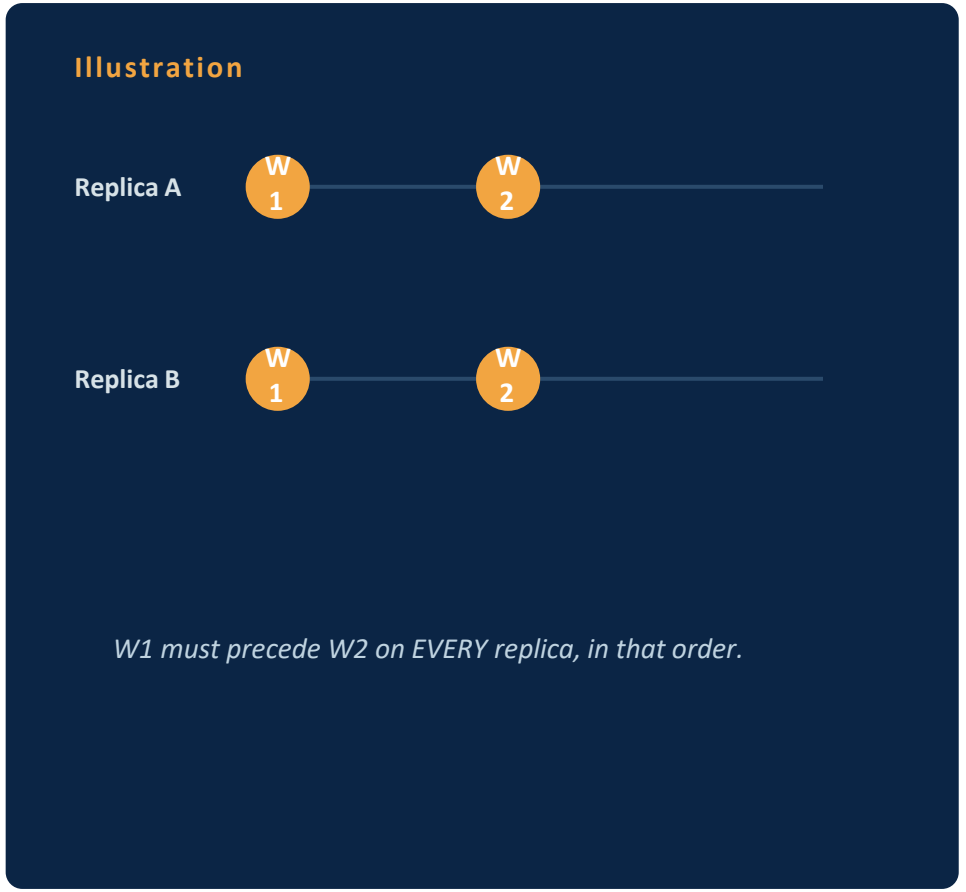


Same process, different replica — R2 must see everything R1 saw (or newer).

Monotonic Writes

- **Guarantee:** a write by a process on item x completes before any successive write on x by the SAME process.
- **In other words:** your own writes are applied everywhere in the order you issued them — replicas can't apply write #2 before write #1.
- **Without this,** a later replica sync could silently overwrite a newer value with an older one.

Example: you update a document's title, then its body. Every replica must apply the title update before the body update — never the reverse.

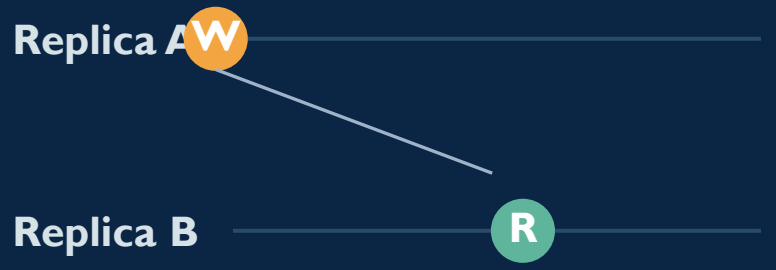


Read Your Writes

- **Guarantee:** the effect of a write by a process on item x is ALWAYS seen by a successive read on x by that SAME process.
- **In other words:** you never fail to see your own most recent update, regardless of which replica answers your read.
- **Classic failure case:** updating a password, then being immediately asked to log in against a stale replica that still has the old one.

Example: you post a comment, refresh the page, and the comment is missing because the read hit a replica that hasn't received your write yet — read-your-writes forbids this.

Illustration



R (on a different replica) MUST reflect W — your own write is never invisible to you.

Writes Follow Reads — and a Summary

- **Guarantee:** a write by a process on x, following a previous read of x BY THAT PROCESS, is guaranteed to take place on the same or a more recent value than the one read.
- **In other words:** your writes are never applied "behind" the state you already observed.
- **Example:** you read a forum post, then reply to it — every replica must apply your reply AFTER the post you read, never before.

Model	Ordering guaranteed
Monotonic Reads	read → read (never see older data)
Monotonic Writes	write → write (your writes stay ordered)
Read Your Writes	write → read (you see your own write)
Writes Follow Reads	read → write (writes build on what you read)

Illustration



Your write (W) must build on the value you already read (R), even on a different replica.

Replication

Deciding how many copies, and where they live

Why Replicate Data?



Performance

Serve reads from a nearby replica — lower latency, higher throughput, load spread across servers.



Reliability

If one replica fails or is partitioned away, others keep serving — no single point of failure.



Scalability

Read-heavy workloads scale horizontally by adding more replicas.



Availability

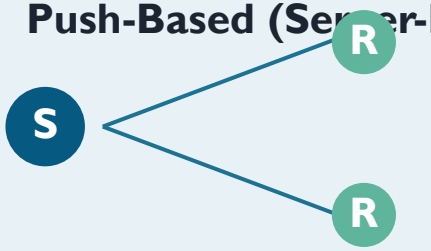
Geographically distributed replicas keep the service reachable during regional outages.

The catch:

every one of these benefits requires KEEPING COPIES UP TO DATE — which costs coordination, bandwidth, and time. Replication and consistency are always in tension; the rest of this section is about managing that tension deliberately.

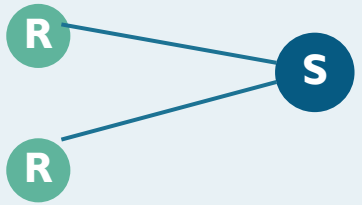
Replication Strategies: Push vs. Pull

Push-Based (Server-Initiated)



- Master pushes updates to replicas as soon as they occur.
- Low read latency — replicas are always (nearly) fresh.
- Wastes bandwidth if replicas are read infrequently.
- Good fit: high read-to-write ratio, need for strong freshness.

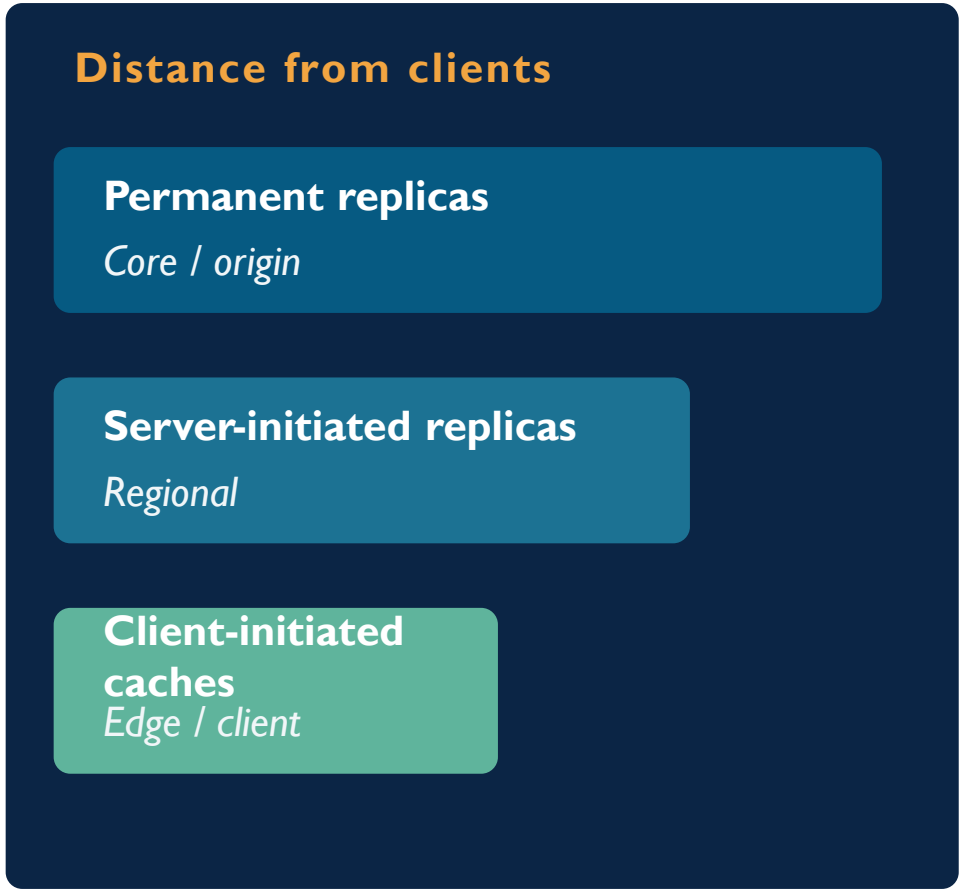
Pull-Based (Client-Initiated)



- Replicas poll the server / cache for updates on demand.
- Saves bandwidth — only fetch when actually needed.
- Reads may return stale data between polls.
- Good fit: low read-to-write ratio, bandwidth-constrained clients.

Placement of Replicas

- **Permanent replicas:** the fixed, initial set of servers that make up the distributed data store (e.g., the primary data centers).
- **Server-initiated replicas:** created dynamically by the system itself — e.g., pushing a popular web page's cache closer to a region seeing a traffic spike.
- **Client-initiated replicas (caches):** created at the client's request, typically temporary — e.g., a browser cache.



Consistency – Replication Trade-offs

Design choice	Gains	Costs
More replicas	Higher availability, read throughput	Harder to keep in sync, more write coordination
Stronger consistency	Simpler application logic, no stale reads	Higher write latency, lower availability under partition
Push propagation	Replicas stay fresh	Bandwidth cost even when unread
Pull propagation	Bandwidth-efficient	Readers may see stale data
Replicas placed near clients	Low read latency	More nodes to keep consistent, cache invalidation complexity

This table is the bridge to the CAP theorem: every row is really trading consistency for availability/performance, or vice-versa.

The CAP Theorem

Why you can't have it all at once

FORMAL STATEMENT

Consistency, Availability, Partition Tolerance



Consistency

Every read receives the most recent write or an error — all nodes see the same data at the same time.



Availability

Every request receives a (non-error) response — without the guarantee that it contains the most recent write.



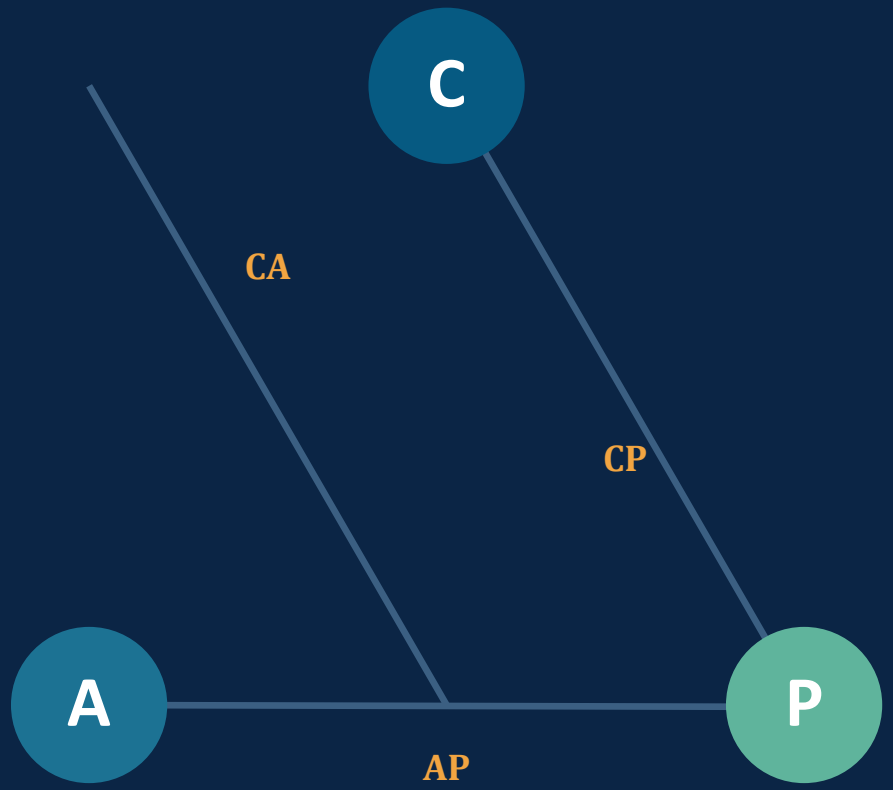
Partition Tolerance

The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network.

CAP Theorem: a distributed data store can provide at most TWO of these THREE guarantees at the same time.

THE CLASSIC PICTURE

The CAP Triangle



Pick any TWO

- **CA:** consistent & available — but can't tolerate a partition (rare in practice; only works on a single site / no network split).
- **CP:** consistent & partition-tolerant — sacrifices availability: some nodes refuse requests during a split.
- **AP:** available & partition-tolerant — sacrifices strict consistency: nodes keep answering, possibly with stale data.

CAP in Practice

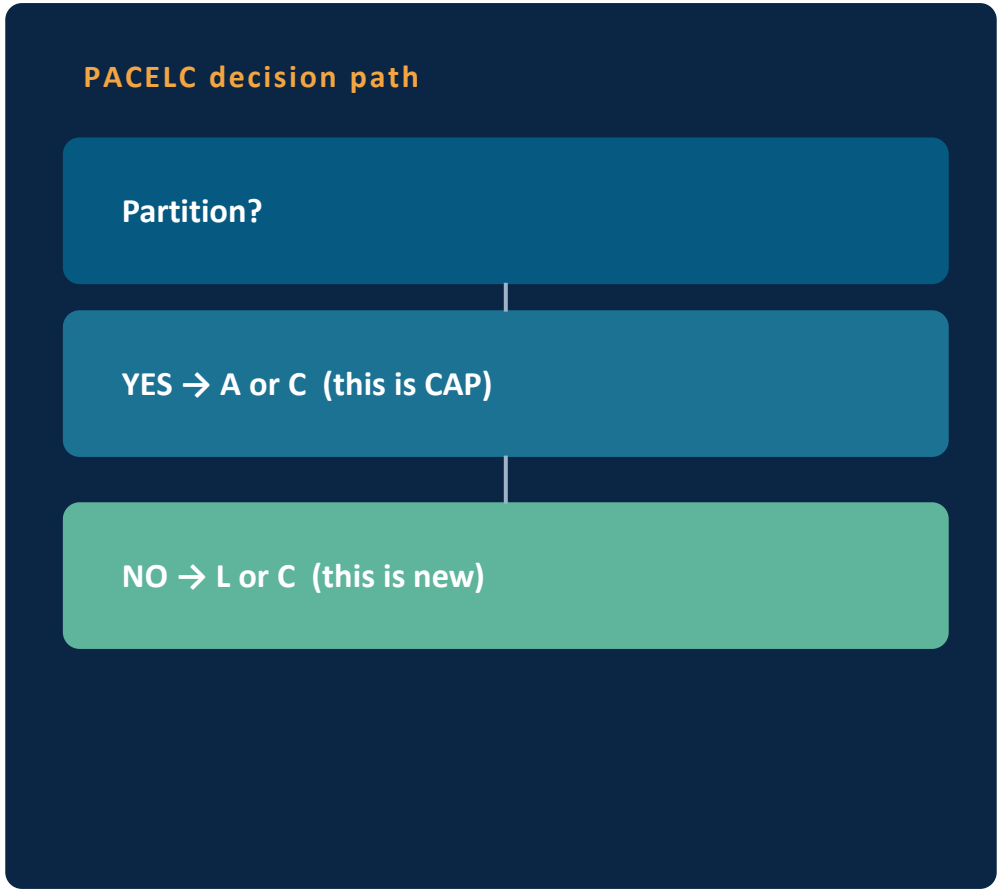
Category	Behavior under partition	Example-style systems
CP systems	Reject/queue requests on the minority side to keep data consistent	Distributed relational DBs with strong quorum writes; ZooKeeper-style coordination services
AP systems	Keep answering reads/writes on both sides; reconcile later	Highly-available key-value / NoSQL stores; DNS; shopping-cart style services
CA systems	Only meaningful without a real network partition	A single-site database cluster on one reliable LAN

Note: real products often let you TUNE the trade-off per operation (e.g., choosing read/write quorum sizes) rather than being purely one category.

PACELC — Extending CAP

- **CAP only describes behavior DURING a partition.** But trade-offs exist even when the network is healthy!
- **PACELC:** if there is a Partition (P), choose between Availability (A) and Consistency (C); Else (E), choose between Latency (L) and Consistency (C).
- **Why it matters:** even a perfectly healthy network forces a choice — strong consistency requires coordination, which adds latency.

Read as: "if Partition → A or C; Else → L or C."



Fault Tolerance & Security

Surviving failures — and staying trustworthy while doing it

Fault Tolerance in Distributed Systems

- **Fault tolerance:** a system's ability to continue operating correctly despite the failure of some of its components.
- **Key terms:** a FAULT is a defect in a component; an ERROR is an incorrect internal state caused by a fault; a FAILURE is when the system stops delivering its expected service.
- **Design goal:** prevent faults from becoming failures — mask them with redundancy, detect them quickly, and recover gracefully.
- **Dependability properties:** availability, reliability, safety, and maintainability are the pillars fault-tolerant design targets.



Types of Failures

Failure type	Description	Severity
Crash failure	A process simply halts, but behaved correctly until it stopped.	Low
Omission failure	A process fails to respond to incoming requests (send/receive omission).	Low–Medium
Timing failure	A response is correct but arrives outside a specified time interval.	Medium
Response failure	The process responds incorrectly — wrong value or wrong control flow.	Medium–High
Arbitrary (Byzantine) failure	A process produces arbitrary output, at arbitrary times — including malicious behavior.	Highest

Increasing severity → as we move down the table, failures become harder to detect and to tolerate.

Redundancy Techniques

Information Redundancy

Extra bits added to data so errors can be detected/corrected (e.g., parity, checksums, error-correcting codes).

Time Redundancy

An action is performed, and if needed, repeated (e.g., transaction retries, timeouts + resend).

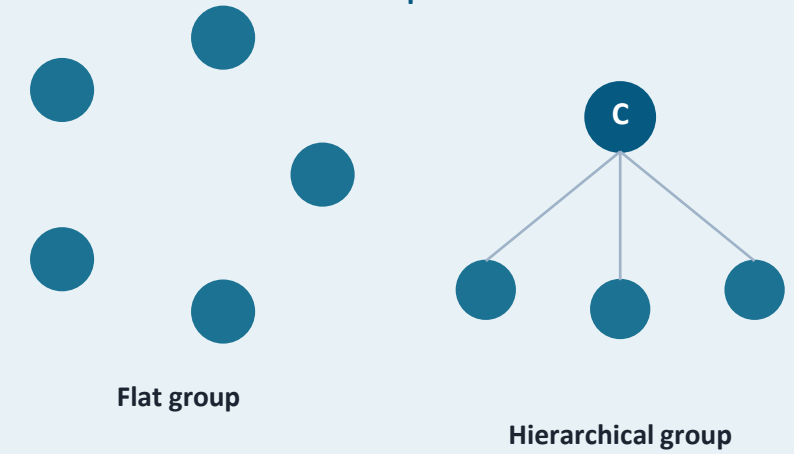
Physical Redundancy

Extra hardware or processes duplicate functionality (e.g., replica servers, redundant network links, RAID disks).

Replication (from Section B) is physical redundancy applied specifically to DATA.

Process Resilience — Overview

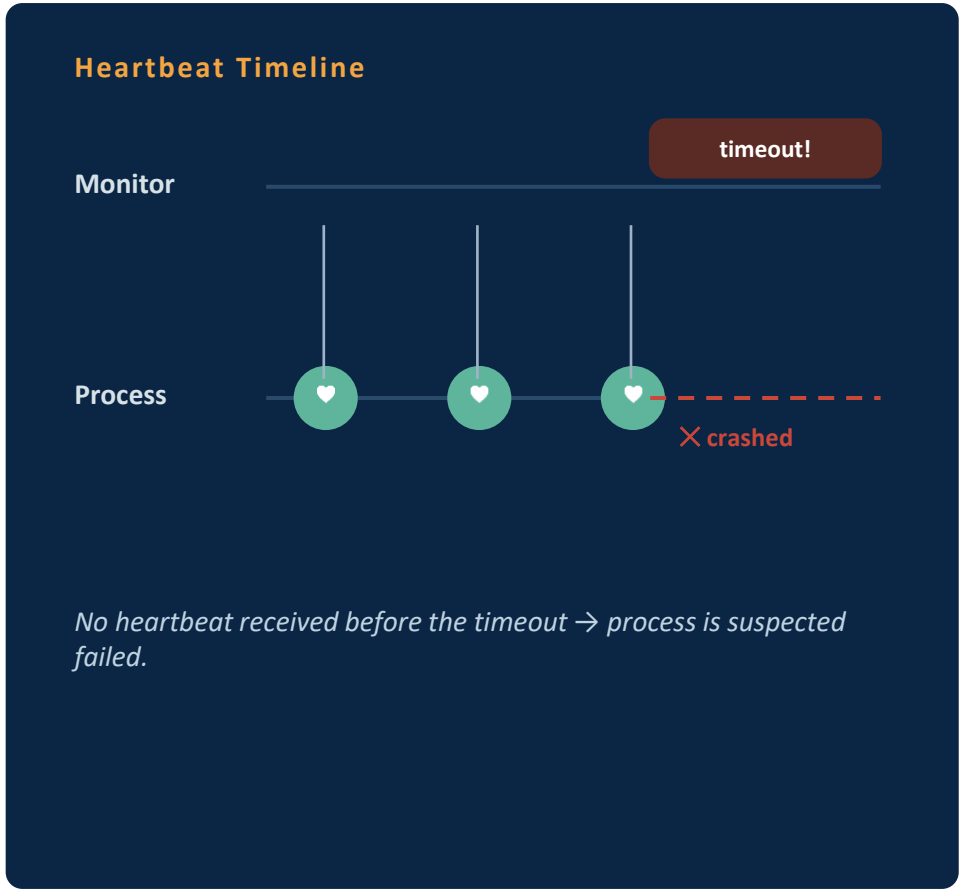
- **Idea:** tolerate the failure of a process by organizing several IDENTICAL processes into a GROUP.
- **Flat groups:** all processes are equal / symmetric — more robust, but decision-making needs coordination (e.g., voting).
- **Hierarchical groups:** one coordinator manages the rest — simpler decisions, but the coordinator is a single point of failure unless it's replaced automatically.
- **Core requirement:** the group must be able to detect when a member has failed — which motivates heartbeat and gossip protocols, next.



Both need a way to know when a member is gone — that's failure detection.

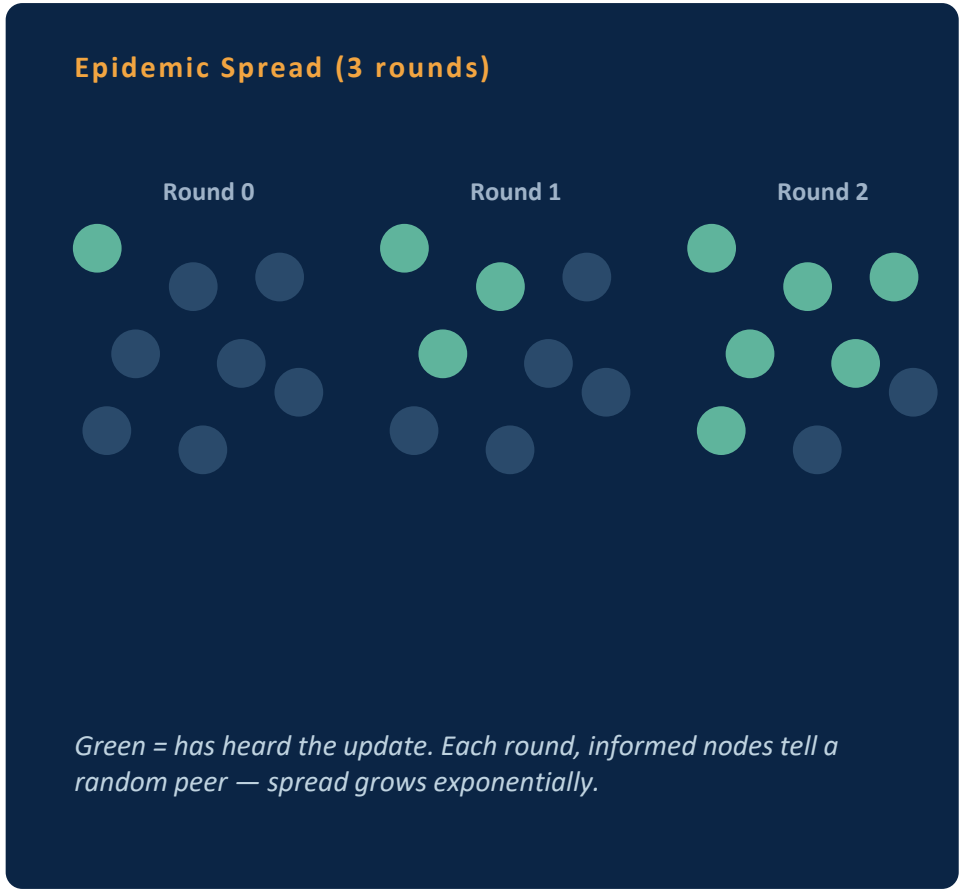
Heartbeat Protocol

- **Idea:** each process periodically sends an “I'm alive” (heartbeat) message to a monitor (or to its neighbors).
- **Detection rule:** if no heartbeat arrives within a timeout window, the process is SUSPECTED failed.
- **Simple and low-overhead,** but a single missed heartbeat (e.g., due to network delay, not an actual crash) can cause a false positive.
- **Tuning trade-off:** shorter intervals detect failures faster but generate more traffic and more false suspicions.



Gossip Protocol

- **Idea:** each process periodically exchanges membership/state information with a few RANDOMLY chosen peers, not a central monitor.
- **Information spreads epidemically:** like a rumor, updates (including failure suspicions) propagate across the whole system in O(log N) rounds.
- **Highly scalable & resilient:** no single point of failure, works well in very large, dynamic clusters.
- **Trade-off:** detection is probabilistic and takes a few rounds to spread — not instantaneous like a direct heartbeat to a monitor.



COMPARING APPROACHES

Heartbeat vs. Gossip

Aspect	Heartbeat	Gossip
Architecture	Centralized (monitor watches processes)	Decentralized (peer-to-peer exchange)
Scalability	Limited — monitor can bottleneck	High — load spread across all nodes
Detection speed	Fast, bounded by timeout	Slower, probabilistic, spreads over rounds
Single point of failure	Yes (the monitor)	No
Best fit	Small-to-medium clusters, simple setups	Very large, dynamic, peer-to-peer systems

SECTION E

Reliable Communication

Making sure messages actually get through — to one, or to many

Reliable Client-Server Communication

- **Point-to-point reliability** typically builds on a reliable transport (e.g., TCP) — but reliable BYTES are not the same as a reliable RPC.
- **Failure at the client:** the client might crash before receiving a reply — orphan computations at the server need cleanup.
- **Failure at the server:** did the server crash BEFORE or AFTER executing the request? The client can't always tell.
- **Semantics choices:** at-most-once, at-least-once, and exactly-once delivery/execution guarantees trade off simplicity vs. correctness.



ONE-TO-MANY

Reliable Group Communication

- **Goal:** reliably deliver a single message to ALL members of a process group, even if some members or the sender fail mid-multicast.
- **Atomic multicast:** a message is delivered to EVERY non-faulty group member, or to NONE of them — no partial delivery.
- **Ordering guarantees layer on top:** FIFO (per-sender order), causal order, and total order (all members see the same order) are common properties requested.
- **Challenge:** the sender might crash partway through sending — the group must agree on whether the message "counts" as delivered.

Ordering Guarantees

FIFO order

Messages from ONE sender arrive in the order sent

Causal order

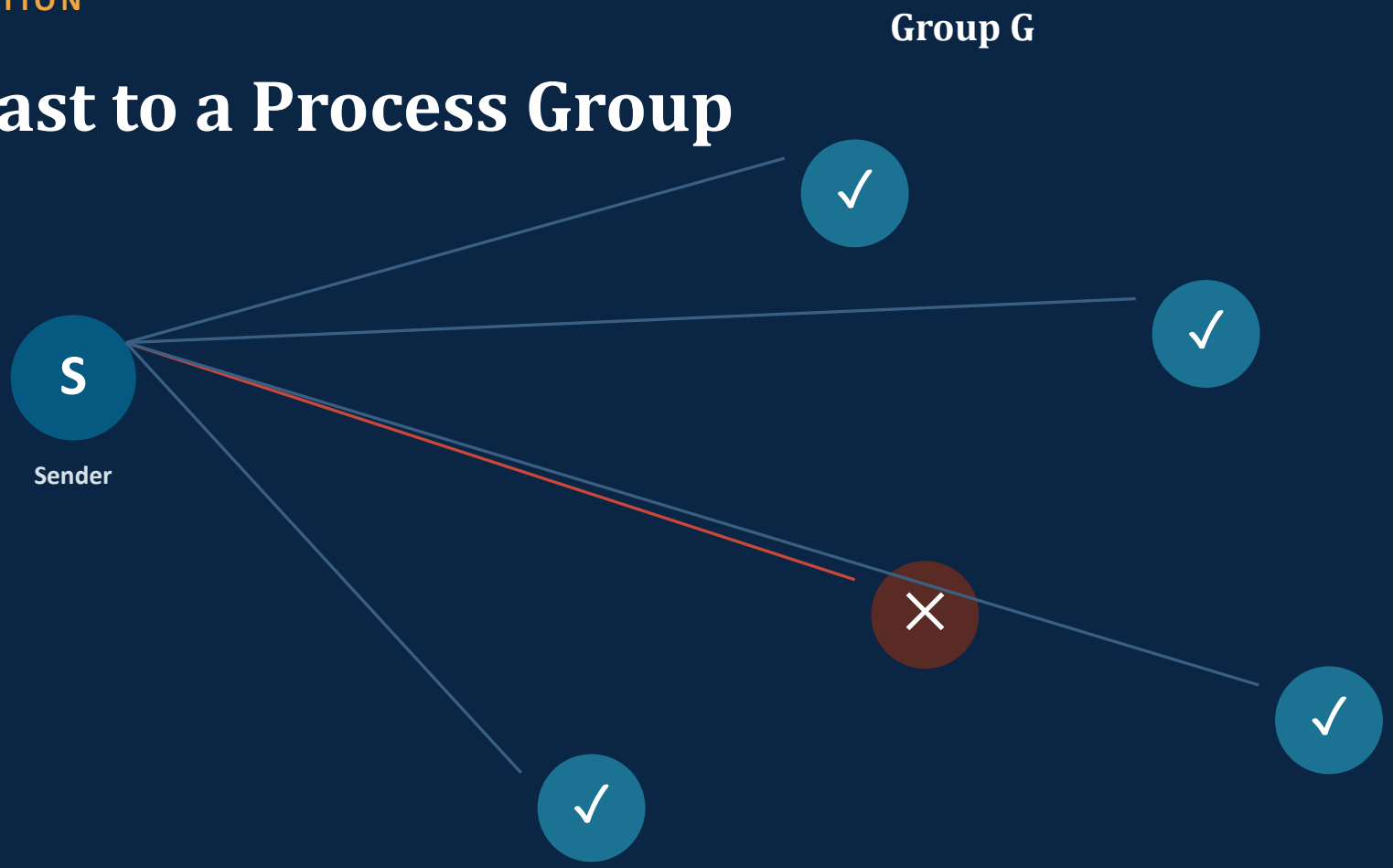
Causally related messages, from any senders, respect happened-before

Total order

ALL members see ALL messages in the SAME order

DEMONSTRATION

Multicast to a Process Group



One member (✗) failed to receive. Atomic multicast: either resend / roll back so ALL non-faulty members end up consistent.

Reliable group communication must tolerate exactly this scenario — a message that reaches most, but not all, members.

Two-Phase Commit (2PC)

- **Goal:** get all participants in a distributed transaction to agree — ALL commit, or ALL abort. No partial commits.
- **Roles:** one COORDINATOR drives the protocol; multiple PARTICIPANTS (cohorts) hold pieces of the transaction.
- **Phase 1 — Voting:** coordinator asks “can you commit?”; each participant votes YES (ready) or NO (abort).
- **Phase 2 — Decision:** if ALL voted YES, coordinator broadcasts COMMIT; if ANY voted NO, it broadcasts ABORT.

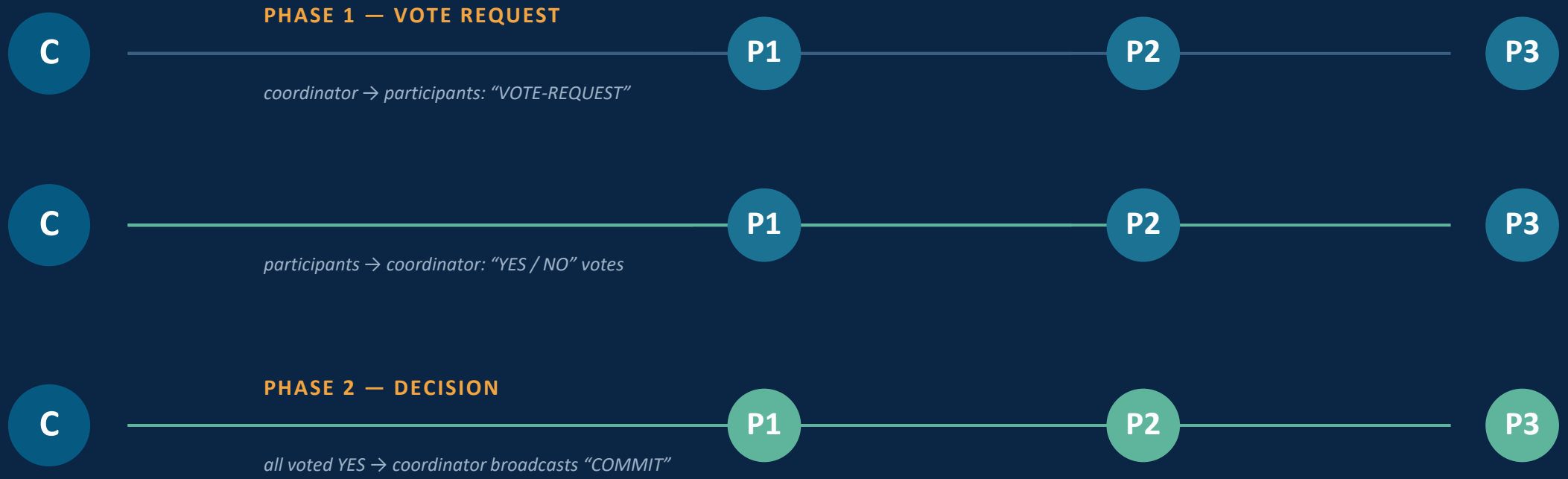
Key Weakness

2PC is a BLOCKING protocol: if the coordinator crashes after participants vote YES but before they hear the final decision, those participants are stuck — they cannot safely commit OR abort on their own, and must wait (potentially holding locks) until the coordinator recovers.

This blocking problem is exactly what Three-Phase Commit (next) is designed to fix.

DEMONSTRATION

2PC — Step by Step



If ANY participant votes “NO” in Phase 1, the coordinator broadcasts “ABORT” instead — every participant rolls back, preserving all-or-nothing atomicity across the whole transaction.

Three-Phase Commit (3PC)

- **Motivation:** eliminate 2PC's blocking problem by adding a buffer phase before the final commit.
- **Phase 1 — Voting:** identical to 2PC — coordinator asks, participants vote YES/NO.
- **Phase 2 — Prepare-to-Commit:** if all voted YES, coordinator sends "PREPARE-TO-COMMIT" (not a final commit yet) — participants acknowledge, now aware a commit IS coming.
- **Phase 3 — Commit:** coordinator sends the final "COMMIT"; participants apply it.
- **Key benefit:** with the extra phase and timeouts, participants can safely make progress even if the coordinator crashes — avoiding indefinite blocking (under non-Byzantine, network-partition-free assumptions).



3PC Steps, and 2PC vs. 3PC

1. **1. VOTE-REQUEST → YES/NO**
2. **2. PREPARE-TO-COMMIT → ACK**
3. **3. COMMIT → applied**

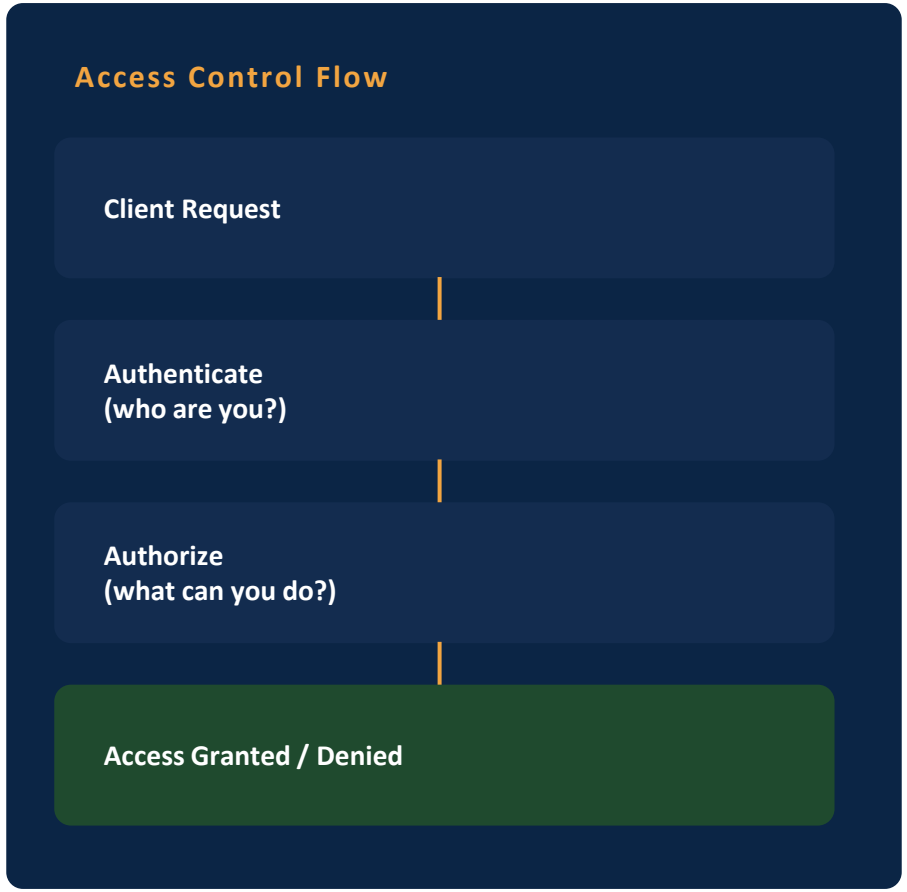
If the coordinator fails after Phase 2 (PREPARE-TO-COMMIT), surviving participants can safely elect a new coordinator and COMMIT — they already know everyone was ready.

2PC	3PC
2 phases	3 phases
Can block on coordinator crash	Avoids blocking (crash-only faults)
Lower message overhead	Higher message overhead
Simpler to implement	More complex, extra timeouts

SECTION G — CLOSING TOPIC

Security & Access Control in Distributed File Systems

- **Authentication:** verifying WHO is making a request — e.g., Kerberos-style ticket systems, mutual TLS certificates.
- **Authorization / access control:** deciding WHAT an authenticated user may do — access control lists (ACLs), capability-based access, or role-based access control (RBAC).
- **Data integrity & confidentiality:** encryption in transit (TLS) and at rest, plus checksums / signatures to detect tampering.
- **Replication adds new risk surface:** every replica is another copy an attacker could target — access control policy must be enforced CONSISTENTLY across all replicas.
- **Auditing:** logging access and changes so breaches or misuse can be detected and traced after the fact.



Summary & Key Takeaways

✓ **Consistency:**

Data-centric models define global rules; client-centric models protect one client's personal view.

✓ **Replication:**

Push/pull propagation and replica placement (permanent, server-, client-initiated) trade freshness for cost.

✓ **CAP theorem:**

Under a network partition, you must choose Consistency or Availability — you always keep Partition tolerance.

✓ **Fault tolerance:**

Faults → errors → failures; redundancy (information, time, physical) masks faults before they become failures.

✓ **Failure detection:**

Heartbeat = centralized & fast; gossip = decentralized & scalable.

✓ **Commit protocols:**

2PC is simple but can block; 3PC adds a phase to avoid blocking under crash-only failures.

✓ **Security:**

Authentication + authorization + encryption must be enforced consistently across every replica.

Next lecture: naming and coordination services, or consensus algorithms (Paxos / Raft) — check the course schedule.